

Lịch buổi tối: 15/8 → 27/8 (bản 2)

Mục tiêu: **lỗi viết bằng C, app chạy được và dùng được.** 13 buổi tối, ~3-4 tiếng mỗi tối.
Đã bỏ: unit test, video demo, phần chuẩn bị kể chuyện. **Đã thêm:** lịch sử tra cứu, từ vựng yêu thích, duyệt theo cấp JLPT, sao chép từ.

1. Ý của thầy: "vẫn nhúng code C trong code C++ được"

Đúng, và đây là điều làm cả kế hoạch bết rưới ro rất nhiều. Ba mức độ, chọn mức nào cũng thoả "dùng C nhiều nhất có thể":

Mức	Cách làm	Khi nào dùng
A	File <code>.c</code> thật, biên dịch bằng trình biên dịch C, bọc <code>extern "C"</code>	Mặc định — nghiêm túc nhất
B	File <code>.cpp</code> nhưng viết theo phong cách C: <code>char*</code> , <code>malloc</code> , <code>sqlite3</code> C API, không <code>class</code> , không <code>STL</code>	Khi mức A gây rắc rối link
C	Trộn tự do	Chỉ ở <code>entrymodel.cpp</code>

Làm mức A, nhưng viết code sao cho tụt xuống mức B lúc nào cũng được.

Cách bảo đảm điều đó: viết code hợp lệ với **cả** C và C++. Chỉ cần nhớ ba chỗ khác nhau:

```
/* 1. malloc trả về void*. C tu chuyên, C++ thì không. Ép kiểu tương minh
   thì cả hai đều biên dịch được. */
DictEntry *p = (DictEntry *)malloc(n * sizeof(DictEntry));

/* 2. Dùng typedef cho struct, dùng cả hai đều gọi ngắn gọn được */
typedef struct DictEntry DictEntry;

/* 3. Hàm không tham số phải viết rõ (void), không được để trống */
int dict_db_version(void);
```

Viết vậy rồi thì lúc nào kẹt, đổi `.c` → `.cpp` và bỏ `LANGUAGES C` khỏi `CMake` là xong — code không phải sửa dòng nào.

Vì sao vẫn nên bắt đầu bằng `.c` thật: trình biên dịch C **cấm** bạn lỡ tay dùng `std::vector` hay `class`. Viết trong `.cpp` thì rất dễ trượt về C++ mà không nhận ra. Ranh giới do máy giữ chắc hơn do người nhớ.
Và giữ `extern "C"` trong header dù ở mức nào — nó vô hại và là chuẩn.

2. Cấu trúc thư mục: hiện tại → đích

Hiện tại

```
D:\FPT\FPTJapan\Dictionary\
├── CMakeLists.txt
├── main.cpp                (app)
├── Main.qml
├── core\
│   ├── entry.h
│   ├── kana.h/.cpp
│   ├── csvreader.h/.cpp
│   ├── common.h/.cpp
│   ├── dictionary.h/.cpp
│   ├── entrymodel.h/.cpp   ← nằm nhầm chỗ
└── tool\importer\main.cpp
```

`entrymodel.*` là C++/Qt nhưng đang nằm trong `core\`. Khi `core\` thành C thuần thì nó phá quy tắc.

Đích

```
D:\FPT\FPTJapan\Dictionary\
├── CMakeLists.txt
├── core\                    ← C THUẦN, không header Qt nào
│   ├── dict_types.h
│   ├── dict_utf8.h/.c
│   ├── dict_kana.h/.c
│   ├── dict_csv.h/.c
│   ├── dict_db.h/.c
│   ├── dict_deinfect.h/.c
│   └── third_party\sqlite3.c, sqlite3.h
├── app\                    ← gom hết phần Qt vào đây
│   ├── entrymodel.h/.cpp
│   ├── main.cpp
│   └── Main.qml
└── tool\importer\main.c
```

Thước đo tự kiểm tra: thấy `#include <Q...>` trong file `.c` là sai chỗ.

Bản đồ từng file

Hiện có	Thành	Ghi chú
core/entry.h	core/dict_types.h	thêm DictEntryList, DictStatus
core/kana.h/.cpp	core/dict_utf8.h/.c + core/dict_kana.h/.c	tách UTF-8 ra riêng
core/csvreader.* + core/common.*	core/dict_csv.h/.c	gộp, đọc theo dòng
core/dictionary.h/.cpp	core/dict_db.h/.c	Qt SQL → sqlite3 C API
core/entrymodel.h/.cpp	app/entrymodel.h/.cpp	giữ C++
main.cpp	app/main.cpp	không đổi nội dung
Main.qml	app/Main.qml	không đổi một dòng
tool/importer/main.cpp	tool/importer/main.c	viết lại

Dùng git mv để giữ lịch sử file.

3. CMakeLists.txt đích

```
cmake_minimum_required(VERSION 3.16)
project(Dictionary VERSION 0.1 LANGUAGES C CXX)

set(CMAKE_C_STANDARD 99)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(QT_QML_GENERATE_QMLLS_INI ON)

find_package(Qt6 REQUIRED COMPONENTS Quick)
qt_standard_project_setup(REQUIRES 6.8)

# ----- Loi C thuan: KHONG link Qt gi ca -----
add_library(dict-core STATIC
    core/dict_types.h
    core/dict_utf8.h      core/dict_utf8.c
    core/dict_kana.h      core/dict_kana.c
    core/dict_csv.h       core/dict_csv.c
    core/dict_db.h        core/dict_db.c
    core/dict_deinflect.h core/dict_deinflect.c
    core/third_party/sqlite3.c
)
target_include_directories(dict-core PUBLIC core core/third_party)

if(UNIX)
    target_link_libraries(dict-core PRIVATE dl pthread m)
endif()

# Tat canh bao cua sqlite3 amalgamation (khong phai code cua minh)
set_source_files_properties(core/third_party/sqlite3.c
    PROPERTIES COMPILE_FLAGS "-w")

# ----- App QML -----
qt_add_executable(appDictionary app/main.cpp)

qt_add_qml_module(appDictionary
    URI Dictionary
    QML_FILES
        app/Main.qml
    SOURCES
        app/entrymodel.h
        app/entrymodel.cpp
)

set_target_properties(appDictionary PROPERTIES
    MACOSX_BUNDLE_BUNDLE_VERSION ${PROJECT_VERSION}
    MACOSX_BUNDLE_SHORT_VERSION_STRING ${PROJECT_VERSION_MAJOR}.${PROJECT_VERSION_MINOR}
    MACOSX_BUNDLE TRUE
    WIN32_EXECUTABLE TRUE
)

target_link_libraries(appDictionary PRIVATE Qt6::Quick dict-core)

install(TARGETS appDictionary
    BUNDLE DESTINATION .
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)

# ----- Importer: C thuan, khong Qt -----
add_executable(importer tool/importer/main.c)
target_link_libraries(importer PRIVATE dict-core)
```

Ba thay đổi so với bản hiện tại

- LANGUAGES C CXX — không có thì CMake không biên dịch file .c
- find_package bỏ Sql — không dùng Qt SQL nữa
- importer dùng add_executable thường, không phải qt_add_executable

Sau khi đổi, importer.exe chạy độc lập không cần DLL Qt nào. Đó là bằng chứng lỗi đã tách sạch.

Nếu tụt xuống mức B: đổi đuôi file thành .cpp, bỏ C khỏi LANGUAGES, bỏ hai dòng CMAKE_C_STANDARD. Không sửa code.

4. Chuẩn code C áp dụng

Đặt tên

Thành phần	Quy ước	Ví dụ
Hàm công khai	dict_<module>_<hành_động>	dict_kana_to_hiragana
Hàm nội bộ file	static, không tiền tố	static int parse_field(...)
Kiểu	PascalCase có tiền tố	DictEntry
Hằng, macro	UPPER_SNAKE	DICT_WORD_LEN
Biến	snake_case	row_count

C không có namespace — tiền tố `dict_` là cách duy nhất tránh đụng tên.

Header

```
#ifndef DICT_KANA_H
#define DICT_KANA_H

#include <stddef.h>
#include "dict_types.h"

#ifdef __cplusplus
extern "C" {
#endif

/* khai bao ham */

#ifdef __cplusplus
}
#endif

#endif /* DICT_KANA_H */
```

Include guard `#ifndef` chứ đừng `#pragma once` — guard là chuẩn C.

Báo lỗi: trả mã, dữ liệu qua tham số ra

```
typedef enum {
    DICT_OK          = 0,
    DICT_ERR_ARG     = -1,
    DICT_ERR_IO      = -2,
    DICT_ERR_NOMEM   = -3,
    DICT_ERR_DB      = -4
} DictStatus;

/* DUNG */
int dict_db_search(DictDb *db, const char *q, int limit, DictEntryList *out);

/* TRANH – mat thong tin vi sao hong */
DictEntryList dict_db_search(DictDb *db, const char *q, int limit);
```

Đây là quy ước chuẩn của C, cũng là cách sqlite3 làm.

Opaque struct cho mọi thứ có trạng thái

```
/* dict_db.h */
typedef struct DictDb DictDb;      /* chỉ khai bao */

/* dict_db.c */
struct DictDb {
    sqlite3 *handle;
    char      last_error[256];
    int       has_user_db;
};
```

Hệ quả: `dict_db.h` **không cần** `#include <sqlite3.h>`. Chi tiết cài đặt không rò ra header.

Sở hữu bộ nhớ — ghi vào header

```
/* out: nguoi goi cap phat struct, ham nay cap phat mang ben trong.
 *      Phai goi dict_entry_list_free(out) khi dung xong,
 *      KE CA khi ham tra ve loi. */
int dict_db_search(DictDb *db, const char *q, int limit, DictEntryList *out);
```

Trong C không compiler nào kiểm tra giúp. Comment **là** hợp đồng.

An toàn chuỗi

Chỉ dùng `snprintf`. Cấm `strcpy`, `strcat`, `sprintf`.

```
int n = snprintf(e->word, sizeof e->word, "%s", src);
if (n < 0 || n >= (int)sizeof e->word)
    fprintf(stderr, "canh bao: cat bot 'word' o dong %d\n", row_no);
```

Viết `sizeof e->word` chứ đừng gõ lại số `64`.

Còn lại

- Không biến toàn cục có thể sửa
- Mỗi nhánh lỗi dọn tài nguyên trước khi `return`
- `static` cho mọi hàm không dùng ngoài file
- `const` cho mọi tham số chỉ đọc
- Ép `(unsigned char)` trước khi thao tác bit trên byte

5. Kích thước buffer (đo trên `data.xlsx`)

Cột	Dài nhất (byte UTF-8)	Chọn
<code>word</code>	42	64
<code>reading</code>	51	64
<code>romaji</code>	26	32
<code>part_of_speech</code>	18	32
<code>meaning</code>	81	128
<code>english</code>	145	192
<code>level</code>	2	8

~600 byte/entry, 100 kết quả = 60KB.

`malloc` dùng cho **danh sách kết quả** (số dòng không biết trước), không dùng cho từng trường — nếu mỗi trường là `char*` thì 100 kết quả cần 800 lần `malloc/free`.

6. Lịch 13 tối

Tối 15/8 — Dọn nhà + UTF-8

Nửa đầu (~1 tiếng): sửa nốt bản C++ rồi mới bỏ nó

- **Bug thật:** bảng `words_fts` được tạo nhưng **chưa nạp dữ liệu**. Thiếu `INSERT INTO words_fts(rowid, meaning, english) SELECT id, meaning, english FROM words`. Không có nó, `searchByMeaning` luôn trả rỗng mà không báo lỗi
- Thêm `return 1` sau các chỗ in lỗi trong `main.cpp` (`db.open`, `q.exec(createSql)`, `QFile::remove`)
- Kiểm tra `headRow >= 0` trước khi gọi `data.at(headRow)` — hiện tại `headRow` khởi tạo `-1` và sẽ crash nếu CSV không có cột `word`
- Chạy lại, ghi vào NOTES: `COUNT(*)`, thời gian import, kết quả `search("あい")`. **Ba con số này là chuẩn đối chiếu cho bản C**
- `git commit + git tag v0.1-cpp` — điểm quay về

Nửa sau: dựng khung C

- Tải `sqlite3 amalgamation` từ `sqlite.org` vào `core/third_party/`
- `git mv` tạo cây thư mục đích, sửa `CMakeLists` theo mục 3
- Viết `core/dict_types.h` — struct, enum lỗi, và ba hàm quản lý danh sách:

```
int dict_entry_list_init(DictEntryList *list, int capacity);
int dict_entry_list_push(DictEntryList *list, const DictEntry *e);
void dict_entry_list_free(DictEntryList *list);
```

`push` tự `realloc` gấp đôi khi đầy — đây là `QVector::append` viết tay.

- Viết `core/dict_utf8.c`:

```
/* Doc mot ky tu tai s, ghi codepoint vao *cp.
 * Tra ve so byte da doc (1..4), hoac 0 neu chuoai hong. */
int dict_utf8_decode(const char *s, uint32_t *cp);

/* Ghi codepoint vao out (can it nhat 4 byte), tra ve so byte da ghi. */
int dict_utf8_encode(uint32_t cp, char *out);
```

Byte đầu quyết định độ dài:

Byte đầu	Số byte	Bit dữ liệu
0xxxxxxx	1	7
110xxxxx	2	11
1110xxxx	3	16
11110xxx	4	21

Byte tiếp theo luôn dạng 10xxxxxx. Kana và kanji đều 3 byte.

Bấy: char có thể là signed → byte 0xE3 thành số âm. Ép (unsigned char) trước mọi phép bit.

Xong khi: decode "こーひー" in ra 3053 30fc 3072 30fc. In hex bằng printf("%04x ", cp) — đừng tin vào mắt nhìn console.

Tối 16/8 — Kana + CSV

core/dict_kana.c:

```
int dict_kana_to_hiragana(const char *src, char *dst, size_t dst_size);
```

Duyệt bằng dict_utf8_decode, codepoint trong [0x30A1, 0x30F6] thì trừ 0x60, mã hoá lại. — (U+30FC) giữ nguyên.

core/dict_csv.c — đọc theo dòng. Dữ liệu của bạn có **0 ô chứa xuống dòng** (đã đếm) nên fgets là đủ, không cần nạp cả file vào RAM.

```
typedef struct DictCsvReader DictCsvReader; /* opaque */

#define DICT_CSV_MAX_FIELDS 16
typedef struct {
    char *fields[DICT_CSV_MAX_FIELDS]; /* tro vao buffer noi bo cua reader */
    int count;
} DictCsvRow;

DictCsvReader *dict_csv_open(const char *path); /* tu cat BOM */
int dict_csv_next(DictCsvReader *r, DictCsvRow *row); /* 1=co, 0=het, <0=loi */
void dict_csv_close(DictCsvReader *r);
int dict_csv_find_column(const DictCsvRow *header, const char *name);
```

Máy trạng thái y hệt bản C++: cờ in_quotes, "" là escape của một dấu nháy.

Mẹo: parse tại chỗ — thay dấu phẩy ngăn ô bằng '\0' rồi cho fields[i] trở vào từng đoạn. Không cấp phát gì thêm.

Xong khi: dict_kana_to_hiragana("こーひー") ra こーひー; parse test.csv ra 4 dòng × 3 ô, dòng 2 giữ nguyên dấu phẩy trong ô.

Tối 17/8 — Ghi database + importer

core/dict_db.c phần ghi:

```
typedef struct DictDb DictDb;

int dict_db_create(const char *path, DictDb **out); /* xoa file cu, tao schema */
void dict_db_close(DictDb *db);
const char *dict_db_last_error(const DictDb *db);

int dict_db_begin(DictDb *db);
int dict_db_commit(DictDb *db);
int dict_db_rollback(DictDb *db);
int dict_db_insert(DictDb *db, const DictEntry *e);
int dict_db_create_indexes(DictDb *db); /* nho COLLATE NOCASE */
int dict_db_create_fts(DictDb *db); /* tao VA nap du lieu */
```

sqlite3 C API cần: sqlite3_open, sqlite3_exec, sqlite3_prepare_v2, sqlite3_bind_text, sqlite3_bind_null, sqlite3_step, sqlite3_reset, sqlite3_finalize, sqlite3_close, sqlite3_errmsg.

Đọc https://sqlite.org/cintro.html — 20 phút.

Bốn bầy - sqlite3_bind_text(stmt, i, s, -1, SQLITE_TRANSIENT) — -1 là "tự đếm tới \0". SQLITE_TRANSIENT bảo sqlite copy chuỗi; SQLITE_STATIC với biến cục bộ là đọc bộ nhớ đã chết - Quên sqlite3_reset() sau mỗi step() trong vòng lặp → chỉ chèn được một dòng - Chỉ số bind bắt đầu từ **1**, không phải 0 - level rỗng phải sqlite3_bind_null, không phải chuỗi rỗng

tool/importer/main.c — đọc dòng, dò header, làm sạch, chèn trong transaction, đo bằng clock().

Xong khi: COUNT(*) = 14.657, khớp con số ghi tối 15/8.

Tối 18/8 — Đọc database + cầu nối ⚠ MỐC KHÔI PHỤC

```
int dict_db_open(const char *path, DictDb **out);
int dict_db_search(DictDb *db, const char *q, int limit, DictEntryList *out);
int dict_db_search_meaning(DictDb *db, const char *q, int limit, DictEntryList *out);
```

Đọc kết quả bằng sqlite3_column_text(stmt, i) — trả const unsigned char*, ép sang const char* rồi snprintf vào struct.

app/entrymodel.cpp — file C++ duy nhất biết về DictEntry:

```
extern "C" {
#include "dict_db.h"
}

DictEntryList list;
if (dict_db_search(m_db, query.toUtf8().constData(), 100, &list) == DICT_OK) {
    for (int i = 0; i < list.count; ++i) {
        Entry e;
        e.word = QString::fromUtf8(list.items[i].word);
        /* ... */
        m_entries.append(e);
    }
}
dict_entry_list_free(&list);    /* BAT BUOC */
```

QString::fromUtf8 chứ không phải QString(...) — cái sau dùng bảng mã hệ thống, chữ Nhật hỏng.

Chiều ngược: query.toUtf8().constData() cho const char*. QByteArray tạm chết ngay sau câu lệnh — đừng lưu con trỏ đó lại.

Tiêu chí dừng: hết tối nay app chưa chạy lại được thì **quay về v0.1-cpp** và báo thầy. Hoặc tụt xuống mức B (đổi .c → .cpp) nếu vướng ở khâu link.

Xong khi: gõ あい trong app ra đúng 10 kết quả như đã ghi tối 15/8.

Tối 19/8 — Tìm kiếm tức thời + thông minh

QML: Timer 250ms debounce; LIMIT 100; trạng thái rỗng có nội dung (chưa gõ → gợi ý, không có kết quả → "thử gõ romaji hoặc nghĩa tiếng Việt"); hiện số kết quả.

C: nhận diện kiểu chuỗi, dùng lại dict_utf8_decode:

```
typedef enum {
    DICT_INPUT_UNKNOWN, DICT_INPUT_KANJI, DICT_INPUT_HIRAGANA,
    DICT_INPUT_KATAKANA, DICT_INPUT_LATIN, DICT_INPUT_VIETNAMESE
} DictInputType;

DictInputType dict_kana_detect(const char *s);
```

Khoảng codepoint: kanji U+4E00–U+9FFF, hiragana U+3041–U+3096, katakana U+30A1–U+30F6. Latin có dấu tiếng Việt (U+1EA0–U+1EF9) → tra FTS5.

Xếp hạng bằng CASE WHEN trong ORDER BY: khớp chính xác → tiền tố → giữa chuỗi.

Giữ đúng một chi tiết từ bản C++: cột word so với chuỗi **gốc**, chỉ reading_hira mới so với bản đã chuyển hiragana. Bạn đã tách :raw / :keyword rồi.

Tối 20/8 — Màn hình chi tiết + sao chép

- StackView: bấm vào từ → push sang trang chi tiết. Nút quay lại + phím Esc. Giữ nguyên kết quả tìm kiếm khi quay lại
- Trang chi tiết: word cỡ lớn, reading, romaji, pos, tất cả nghĩa Việt dạng danh sách đánh số, nghĩa Anh, level
- Tách meaning theo "; " bằng QString::split ở tầng bridge — đừng viết strtok_r trong C chỉ để tách chuỗi hiển thị
- Nút sao chép:** copy word hoặc cả khối thông tin vào clipboard. QML dùng TextEdit ẩn với selectAll() + copy(), hoặc thêm Q_INVOKABLE void copyToClipboard(const QString&) trong EntryModel dùng QApplication::clipboard()
- Phím tắt: Ctrl+F vào ô tìm, Esc xoá/quay lại, ↑ ↓ chọn, Enter mở chi tiết

Sao chép là chức năng nhỏ nhưng người học tra từ điển dùng liên tục — đáng 20 phút.

Tối 21/8 — Lịch sử + yêu thích

Đây là chức năng người dùng thật cần nhất, và cũng là **quyết định kiến trúc đáng kể lúc phỏng vấn**.

Nguyên tắc: tách DB hệ thống và DB người dùng. dictionary.db chỉ đọc, nằm cạnh exe. user.db ghi được, nằm ở QStandardPaths::AppDataLocation — cập nhật app không mất dữ liệu người dùng, và app cài ở Program Files vẫn ghi được.

Kỹ thuật: dùng ATTACH DATABASE để JOIN được giữa hai file.

```
ATTACH DATABASE '<duong_dan>/user.db' AS userdb;

CREATE TABLE IF NOT EXISTS userdb.history (
    entry_id INTEGER PRIMARY KEY,
    viewed_at INTEGER NOT NULL
);
CREATE TABLE IF NOT EXISTS userdb.favorites (
    entry_id INTEGER PRIMARY KEY,
    added_at INTEGER NOT NULL
);

/* Ghi lịch sử, tra cứu lại thì cập nhật thời gian */
INSERT INTO userdb.history(entry_id, viewed_at) VALUES(?, ?)
ON CONFLICT(entry_id) DO UPDATE SET viewed_at = excluded.viewed_at;

/* Lay lịch sử xem đầy đủ thông tin từ */
SELECT w.* FROM userdb.history h
JOIN words w ON w.id = h.entry_id
ORDER BY h.viewed_at DESC LIMIT ?;
```

API C:

```
int dict_db_attach_user(DictDb *db, const char *user_db_path);
int dict_db_add_history(DictDb *db, int entry_id);
int dict_db_list_history(DictDb *db, int limit, DictEntryList *out);
int dict_db_clear_history(DictDb *db);
int dict_db_toggle_favorite(DictDb *db, int entry_id);
int dict_db_is_favorite(DictDb *db, int entry_id, int *out);
int dict_db_list_favorites(DictDb *db, int limit, DictEntryList *out);
```

QML: nút sao ở trang chi tiết; ô tìm kiếm rỗng thì hiện lịch sử thay vì màn hình trắng; một trang xem danh sách yêu thích; nút xoá lịch sử.
Xong khi: tắt app mở lại vẫn còn dữ liệu, và dictionary.db không bị ghi vào (kiểm tra bằng thời gian sửa file).

Tối 22/8 — Bộ lọc + duyệt theo cấp

- ComboBox lọc theo part_of_speech (17 giá trị) và level
- Mục "Chưa phân loại" dùng level IS NULL (9.044 dòng)
- **Duyệt theo cấp JLPT:** ô tìm rỗng + chọn N3 → liệt kê từ N3. Dùng lại đúng truy vấn lọc, chỉ bỏ điều kiện từ khoá. Rẻ mà hữu ích thật cho người ôn thi
- Nút xoá hết bộ lọc, hiện số kết quả sau khi lọc
- Ghép mệnh đề WHERE động bằng chuỗi, nhưng **giá trị luôn bind bằng ?**

Bây: = NULL không bao giờ đúng, phải IS NULL.

Tối 23/8 — Deinflection phần 1

```
typedef struct {
    const char *from; /* duoi dang chia */
    const char *to; /* duoi dang goc */
    int type; /* nhom dong tu */
} DeinflectRule;

static const DeinflectRule RULES[] = {
    { "ました", "る", DICT_VERB_ICHIDAN },
    /* ... */
};

/* out: mảng do nguoi goi cap phat, toi da max phan tu.
 * Tra ve so ung vien, hoac <0 neu loi. */
int dict_deinflect(const char *word, char out[][DICT_WORD_LEN], int max);
```

Mảng static const ở phạm vi file — không cấp phát động. Đây là chỗ C gọn hơn C++.
So khớp đuôi bằng memcmp trên byte cuối — đuôi là chuỗi byte cố định, không cần decode UTF-8.
Nối vào luồng tìm: tra trực tiếp trước, không thấy mới lùi chia. Hiện "Dạng gốc của 食べました".

Đối chiếu: <https://www.sljfaq.org/cgi/verb-deinflect.cgi>

Xong khi: 食べました → 食べる, 見ない → 見る, 行きます → 行く.

Tối 24/8 — Deinflection phần 2

Âm tiện (音便): つて → う/つ/る, んで → ん/ぶ/む, いて → く, いで → ぐ. Một đuôi ra nhiều gốc.
Tính từ đuôi い: かった, くない, くて, ければ.
Thể khả năng, bị động, sai khiến, mệnh lệnh.
Đệ quy giới hạn độ sâu 5 (食べられなかった cần lùi 3 bước). Giữ danh sách đã thử để chống lặp vô hạn.
Lọc ứng viên: chỉ giữ cái thực sự có trong từ điển.
Xong khi: thử 20 dạng chia, đúng ít nhất 15. Ghi con số vào NOTES.md.

Tối 25/8 — Dọn dẹp + kiểm tra bộ nhớ

Không có unit test thì AddressSanitizer là lưới an toàn duy nhất. Đừng bỏ.

```
target_compile_options(dict-core PRIVATE
    $<$<CONFIG:Debug>:-fsanitize=address -fsanitize=leak>)
target_link_options(dict-core PUBLIC
    $<$<CONFIG:Debug>:-fsanitize=address -fsanitize=leak>)
```

Chạy importer trên đủ 14.657 dòng — vòng lặp lớn nên leak lộ ra ngay. Rồi chạy app, tra vài chục từ, thêm/bỏ yêu thích, xem có leak trong dict_db_search không.

- Mọi malloc có free chưa? Mọi nhánh lỗi có dọn trước khi return chưa?
- File .c nào còn #include <Q...>?
- Đổi tên biến khó hiểu, xoá code chết, tách file quá dài
- .gitignore: thư mục build, *.db, *.csv
- Đẩy GitHub, clone về thư mục mới thử build

Tối 26/8 — Đóng gói + README

- Build Release
- windeployqt chép DLL Qt cạnh appDictionary.exe, thử trên máy chưa cài Qt
- importer.exe không cần DLL nào — kiểm tra lại, đó là bằng chứng lỗi tách sạch
- README: mô tả 1 dòng, ảnh chụp màn hình, **sơ đồ ranh giới C/C++**, hướng dẫn build, nguồn dữ liệu
- Nêu lý do để .db bên ngoài thay vì nhúng vào binary, và lý do tách user.db khỏi dictionary.db

Tối 27/8 — Dự phòng

Chắc chắn có việc bị trễ. Tối này dành cho nó.

Nếu không nợ gì, theo thứ tự ưu tiên: 1. Unit test cho dict_utf8 và dict_deinfect — C thuần với assert.h, không cần framework. Đây là thứ đáng thêm nhất nếu còn thời gian 2. Flashcard ôn tập từ danh sách yêu thích 3. Luồng nền QtConcurrent cho truy vấn

7. Thứ tự cắt nếu bị chậm

Cắt từ dưới lên:

Ưu tiên	Hạng mục
1 (không cắt)	Chuyển lỗi sang C, app chạy được
2	Tìm kiếm tức thời + thông minh
3	Màn hình chi tiết + sao chép
4	Đóng gói + README
5	Dọn dẹp + kiểm tra bộ nhớ
6	Lịch sử + yêu thích
7	Deinfection phần 1
8	Deinfection phần 2
9	Bộ lọc + duyệt theo cấp

Chậm 1 tối → bỏ mục 9. Chậm 3 tối → bỏ 8 và 9.

8. Nghiệm thu cuối

- [] Không file .c nào include header Qt
- [] importer.exe chạy được mà không cần DLL Qt
- [] COUNT(*) = 14.657, không dòng nào reading rỗng
- [] EXPLAIN QUERY PLAN không câu nào hiện SCAN words
- [] searchByMeaning("yêu") ra kết quả — bảng FTS5 đã được nạp
- [] Tắt app mở lại, lịch sử và yêu thích còn nguyên

- [] `dictionary.db` không bị ghi vào khi dùng app
- [] AddressSanitizer sạch khi chạy importer đủ 14.657 dòng
- [] Clone về thư mục mới build được
- [] Người khác giải nén file release là chạy được